

DESIGN AND ANALYSIS ALGORITHMS

LECTURE 5

Trial and Error Methods
(Các phương pháp thử sai)

Review the Previous Lesson

- ❑ Divide and conquer method
 - Idea
 - D&C algorithm diagram
 - D&C algorithm analysis: correctness, time complexity
 - ❑ Other type of simplify and conquer methods
 - Decrease and conquer
 - Transform and conquer
-

Lecture outline

- ❑ Introduction
- ❑ Exhaustive method
- ❑ Backtracking method
- ❑ Branch and bound method
- ❑ Exercises

Reference link and book chapter

Prof. Xukai Zou, Computer Science Dept.,
Indiana University Purdue University Indianapolis

<http://cs.iupui.edu/~xzou/>

Prof. Pasi Fränti, School of Computing,
University of Eastern Finland

<http://cs.uef.fi/pages/franti/asa/>

Anany's book, chapter 3, section 12.1, 12.2

Introduction

-
- ✓ Trial and Error Methods
 - ✓ Brute Force Search
 - ✓ Exhaustive Search
-

Trial and Error Methods



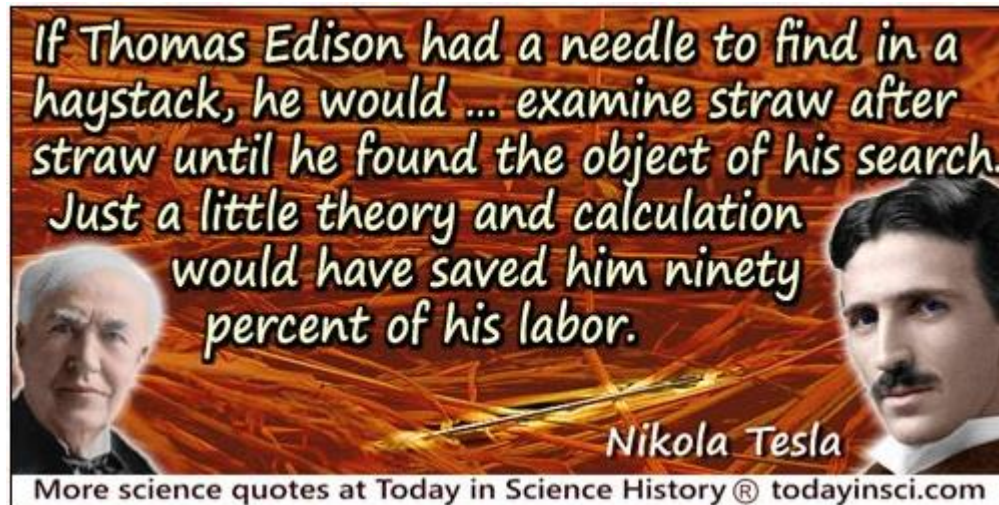
Ảnh: Nguồn internet

Trial and error methods

- ❑ Wiki: Trial and error is a fundamental method of problem-solving.[1] It is characterized by repeated, varied attempts which are continued until success,[2] or until the practicer stops trying.
 - Duyệt tất cả các phương án tồn tại nghiệm của bài toán để xác định nghiệm đúng hoặc đủ tốt.
 - ❑ Pros and Cons:
 - **Pros**: Solution will be found
 - **Cons**: Time consuming
-

Brute Force Search

- “Edison and Needle In A Haystack”



https://todayinsci.com/QuotationsCategories/N_Cat/NeedleInAHaystack-Quotations.htm

- **Brute force** is a straightforward approach to solving a problem, usually directly based on the problem statement and definitions of the concepts involved.

Brute Force Search

□ Brute force template:

Solution domain (tập khả năng) $D = \{(x_1, x_2, \dots, x_n)\}$

$x_i \in D_i$, where $D_i = \{d_{ij} \mid j = 1 \dots m_i\}$ – finite sets

Rule for deriving solution:

$f : D \rightarrow \{true, false\}$

Solution: $x = (x_1, x_2, \dots, x_n)$ such $f(x) = true$

Brute Force Search

□ Brute force diagram:

```
BruteForce(D) ≡  
  for ( $x_1 = d_{11} \dots d_{1m_1}$ )  
    for ( $x_2 = d_{21} \dots d_{2m_2}$ )  
      .....  
      for ( $x_n = d_{n1} \dots d_{nm_n}$ )  
        if ( $f(x_1, x_2, \dots, x_n)$ )  
          <Solution  $x = (x_1, x_2, \dots, x_n)$ >;  
        end.
```

Brute Force Search

□ Brute force example:

■ ***Bài toán gà ,chó***

Vừa gà vừa chó; Bó lại cho tròn;
Ba mươi sáu con; Một trăm chân chẵn
Hỏi số gà, số chó?

■ Phân tích:

Gọi x, y là số gà, chó: $x \in D_1 = \{1, 2, \dots, 36\}$, $y \in D_2 = \{1, 2, \dots, 36\}$

Không gian nghiệm của bài toán: $D = \{(x, y)\}$

Quy tắc xác định lời giải: $f(x, y) = (x + y = 36) \text{ and } (2x + 4y = 100)$

Nghiệm của bài toán $x = (x, y)$ sao cho $f(x, y) = \text{true}$

Brute Force Search

❑ Brute force example: Bài toán gà ,chó

■ Diagram:

ChickenDog $\equiv$

```
for (x=0 .. 36)
```

```
  for (y=0 .. 36)
```

```
    if ( (x+y) = 36 & (2x + 4y) = 100 )
```

```
      <output (x,y)>;
```

```
end.
```

■ Time complexity: $O(n^2)$, $n=36$ – depend on problem space

■ Improve:

- Reduce problem space: $D=\{(x)\}$, $x \in D_1 = \{1,2,\dots,25\}$
- Solution rule: $f(x) = (4x+2(36-x) = 100)$
- Time complexity: $O(n)$, $n=25$

Brute Force Search

❑ Other problems:

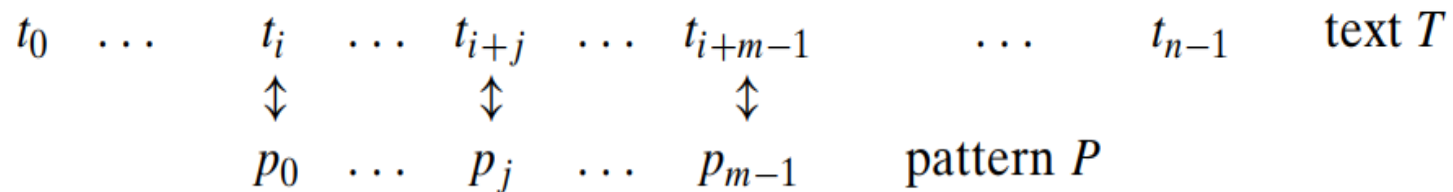
- Selection sort and Bubble sort – Brute force
- Searching – Brute force :
 - Sequential search (tìm kiếm phần tử trong dãy)
 - String matching (khớp xâu)
 - Closest pair of point (cặp điểm gần nhất)
 - Convex hull (bao lồi)

Anany's book, section 3.1, 3.2, 3.3

Brute Force String Matching

□ String Matching Problem

- Input: two strings $T(0..n-1)$ - text, $P(0..m-1)$ – pattern
- Output: i , such $P(0..m-1) = T(i..i+m-1)$; -1 otherwise

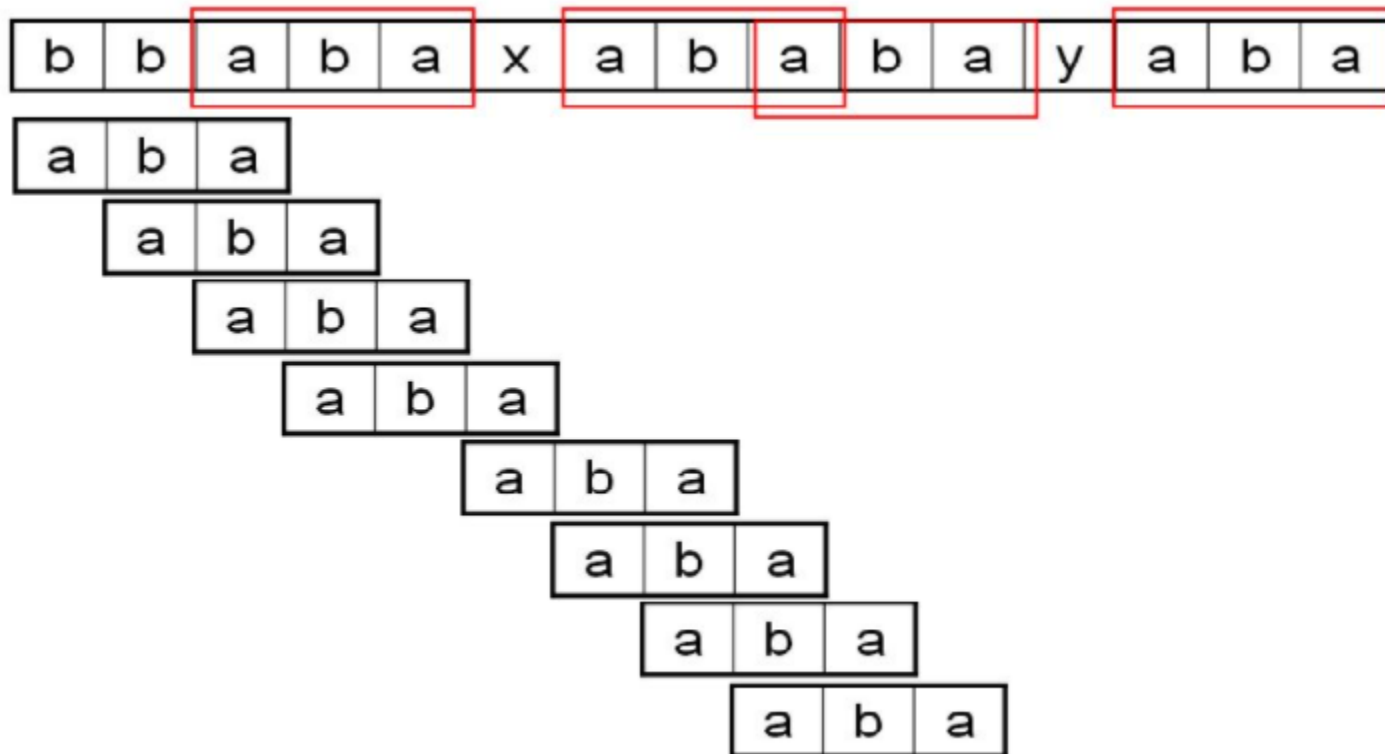


- Brute Force:
 - Align the corresponding pairs of characters of pattern and text from position i ($i=0$ at first).
 - If all m characters are matched - return i
 - If otherwise shift the pattern one position to the right and repeat the comparisons.

Brute Force String Matching

□ String Matching Problem

■ Brute Force:



Brute Force String Matching

□ String Matching Problem

- Brute Force algorithm

```
for  $i \leftarrow 0$  to  $n - m$  do  
     $j \leftarrow 0$   
    while  $j < m$  and  $P[j] = T[i + j]$  do  
         $j \leftarrow j + 1$   
    if  $j = m$  return  $i$   
return  $-1$ 
```

- Time complexity:
 - Naïve: $O(m(n-m+1)) = O(mn)$
 - Improve: $O(?)$

Brute Force String Matching

- String Matching Problem
 - Some improved algorithm

String search algorithm	Time complexity for	
	preprocessing	matching
Naïve	0 (none)	$\Theta(n \cdot m)$
Rabin-Karp	$\Theta(m)$	avg $\Theta(n + m)$ worst $\Theta(n \cdot m)$
Finite state automaton	$\Theta(m \Sigma)$	$\Theta(n)$
Knuth-Morris-Pratt	$\Theta(m)$	$\Theta(n)$
Boyer-Moore	$\Theta(m + \Sigma)$	$\Omega(n/m), O(n)$
Bit based (approximate)	$\Theta(m + \Sigma)$	$\Theta(n)$

m – the length (size) of the pattern; n – the length of the searched text
 $|\Sigma|$ is the size of the alphabet

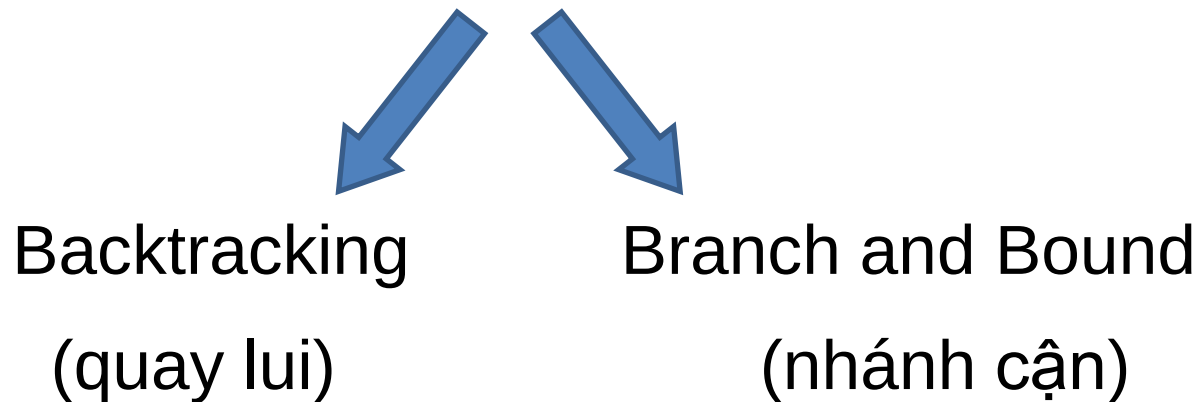
Exhaustive Search

- ❑ Combinatorial problems: finding a grouping, ordering, or assignment of a discrete, finite set of objects that satisfies given condition - Exhaustive search
- ❑ **Exhaustive search:** It suggests generating each and every element of the problem domain, selecting those of them that satisfy all the constraints, and then finding a desired
- ❑ Examples
 - Traveling Salesman Problem (Bài toán người đưa hàng)
 - Knapsack (Xếp ba lô)
 - Assignment Problem (Phân công công việc)
 - ...

Anany's book, section 3.4

Brute Force/Exhaustive searching

- ❑ Limitation of BF/Exhaustive search: large searching space
- ❑ Need method to reduce searching space – only consider available instance.



Backtracking

-
- ✓ Introduction
 - ✓ Template and Diagram of Backtracking
 - ✓ Examples
 - ✓ Applications
-

Introduction

□ Ý tưởng

- Quay lui (Back tracking): Theo nguyên tắc vét cạn, nhưng chỉ xét những trường hợp khả quan.
 - Dùng để giải bài toán liệt kê các cấu hình:
 - Mỗi cấu hình được xây dựng bằng cách xác định từng phần tử
 - Mỗi phần tử được chọn bằng cách thử các khả năng có thể.
 - Tại mỗi bước, nếu có một lựa chọn được chấp thuận thì ghi nhận lại lựa chọn này và tiến hành các bước thử tiếp theo. Còn ngược lại không có lựa chọn nào thích hợp thì quay lại bước trước $\Rightarrow$ quay lui.
-

Introduction

□ Mô hình

Không gian nghiệm của bài toán (tập khả năng) $D=\{(x_1, x_2, \dots, x_n)\}$ gồm các cấu hình liệt kê có dạng $(x_1, x_2, \dots, x_n)$ cần được xây dựng:

- Cho x_1 nhận lần lượt các giá trị có thể. Với mỗi giá trị thử gán cho x_1 thì:
- Cho x_2 nhận lần lượt các giá trị có thể. Với mỗi giá trị thử gán cho x_2 thì xét khả năng chọn $x_3, \dots, x_n$, $\Rightarrow$ cấu hình tìm được $(x_1, x_2, \dots, x_n)$.
- Tại mỗi bước i : Xây dựng thành phần x_i
 - Xác định x_i theo khả năng v .
 - Nếu $i = n$ thì ta có được một lời giải, ngược lại thì tiến hành bước $i+1$ để xác định x_{i+1} .
 - Nếu không có một khả năng nào chấp nhận được cho x_i thì lại lùi lại bước trước để xác định lại thành phần x_{i-1} .

Introduction

□ Lược đồ phương pháp

```
Try(i)  ≡ //Sinh thành phần thứ i của cấu hình
    for (v thuộc tập khả năng thành phần nghiệm  $x_i$ )
        if (  $x_i$  chấp nhận được giá trị v)
             $x_i = v$ ;
            <Ghi nhận trạng thái chấp nhận v>;
            if (i = n) //đủ n thành phần của cấu hình đã xác định
                <ghi nhận nghiệm>;
            else //lời gọi sinh thành phần tiếp theo của cấu hình
                Try (i + 1)
            <Khôi phục trạng thái chưa chấp nhận v>;
        endif;
    endfor
End.
```

Example 1

□ Bài toán: Liệt kê dãy nhị phân độ dài n

■ Phân tích:

- Dãy nhị phân $(x_1x_2\cdots x_n)$ trong đó $x_i=0,1$
- Dùng giải thuật $\text{Try}(i)$ để sinh giá trị x_i
- Nếu $i=n$ thì in giá trị nghiệm, ngược lại sinh tiếp x_{i+1} bằng $\text{Try}(i+1)$

■ Lược đồ:

$\text{Try}(i) \equiv$

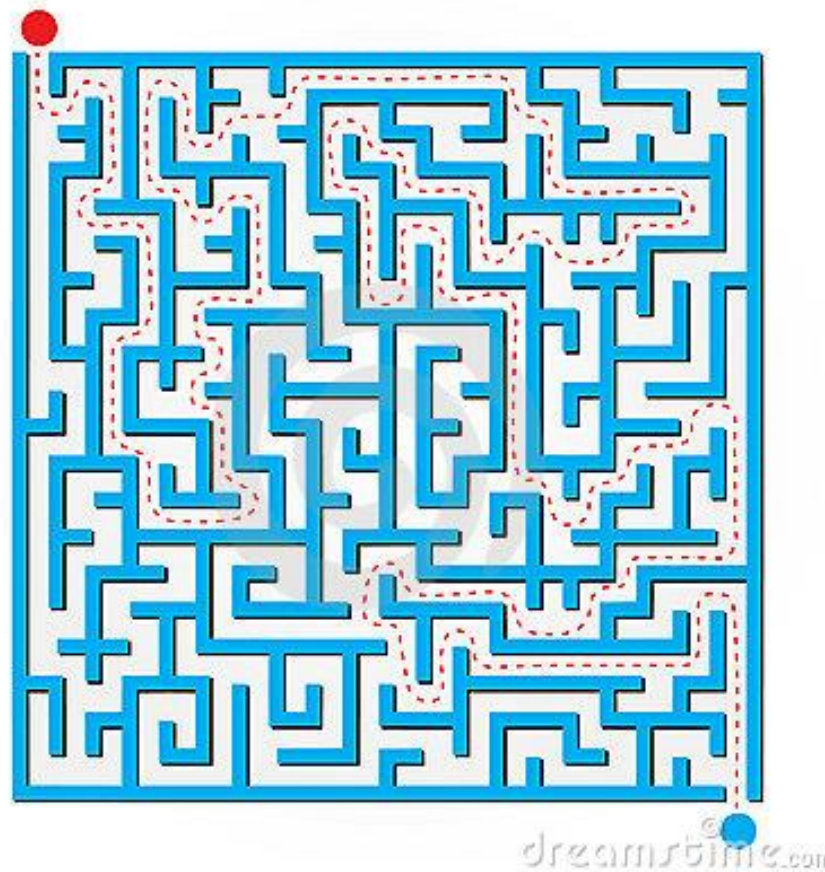
```
for (v=0..1)  //v nhận giá trị 0 hoặc 1
     $x_i = v;$ 
    if (i= n) printResult (x);
    else Try(i+1) ;
endfor;
```

End.

Lời gọi ban đầu **Try(1)**

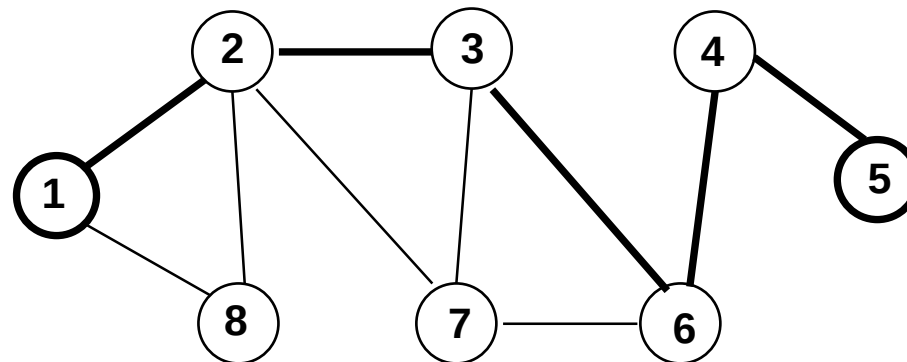
Example 2

- ## 📄 Bài toán: Tìm đường đi trong mê cung



Example 2

- Bài toán: Tìm đường đi trong mê cung
 - Phân tích: Xét mê cung như một đồ thị vô hướng $G=(V,E)$
 - Mỗi phòng của mê cung là một nút của đồ thị
 - Mỗi đường đi giữa các phòng là một cạnh nối giữa các nút của đồ thị
 - Đỉnh xuất phát: $S \in V$, đỉnh kết thúc $F \in V$
 - Đường đi tìm được là dãy $S=x_1, x_2, \dots, x_k=F$ với $x_i \in V, (x_i, x_{i+1}) \in E$



Ví dụ: $S=1, F=5 \in V \rightarrow$ Đường đi: $S = 1 - 2 - 3 - 6 - 4 - 5 = F$

Example 2

□ Bài toán: Tìm đường đi trong mê cung

■ Ý tưởng thuật toán (dùng quay lui)

- Từ đỉnh xuất phát, hình thành dần các đỉnh trên đường đi tới đỉnh đích.
- Dùng giải thuật $\text{Try}(i)$ để sinh giá trị x_i
 - Đỉnh v được chấp nhận cho x_i nếu v chưa được đi qua, có cạnh (x_{i-1}, v)
 - Nếu $x_i = F$ thì kết thúc, in đường đi tìm được
 - Ngược lại sinh tiếp x_{i+1} bằng $\text{Try}(i+1)$
- Lời gọi ban đầu
 - $x_1 = S$
 - $\text{Try}(2)$

Example 2

□ Bài toán: Tìm đường đi trong mê cung

■ Mô tả dữ liệu

○ Input: Đồ thị G , đỉnh S, F

- n đỉnh của đồ thị được đánh số từ $1..n$

- Ma trận kề $M = (m_{ij})$ cỡ $n \times n$ của đồ thị

$m_{ij} = 1$ nếu có cạnh nối đỉnh i với đỉnh j
 $= 0$ nếu ngược lại

○ Output: đường đi $S = x_1, x_2, \dots, x_k = F$

○ Sử dụng biến:

Mảng: Boolean $Daqua[i:1..n]$ đánh dấu đỉnh i đã được đi qua hay chưa?

Khởi tạo $Daqua[i:1..n] = false$

Example 2

□ Bài toán: Tìm đường đi trong mê cung

■ Lược đồ

Try(i) $\equiv$

```
for (v=1..n)  //duyet qua các đỉnh
    if ((m[xi-1,v]=1) & (not Daqua[v])) //v chấp nhận được
        xi = v;
        Daqua[v] = true; //ghi nhận trạng thái đã chọn v
        if (xi= F) printResult (x1,x2,...,xi);
        else Try(i+1) ;
        Daqua[v] = false; //khôi phục trạng thái chưa chọn v
    endif;
endfor;
```

End.

Lời gọi ban đầu: x₁=S;

Try(2) ;

Example 3

□ Bài toán: Phân công công việc

- Có n người với n công việc, người thứ i làm việc thứ j với năng suất $s[i,j]$.
Tìm phương án phân công việc cho mỗi người để tổng năng suất lớn nhất.
- Ý tưởng thuật toán (dùng quay lui)
 - Một phương án phân chia công việc $\equiv$ 1 cấu hình liệt kê (người-việc)
 - Dùng giải thuật Try(i) để sinh giá trị w_i là công việc phân cho người i
 - Công việc v được chấp nhận cho w_i nếu v chưa được phân công, và người i có thể làm việc v .
 - Nếu $i=n$ thì kết thúc, xử lí phương án tìm được:
 - Tính tổng hiệu suất của phương án
 - So sánh với hiệu suất max đã tính được. Cập nhật nếu cần.
 - Ngược lại ($i < n$) sinh tiếp w_{i+1} bằng Try($i+1$)

Example 3

□ Bài toán: Phân công công việc

- Có n người với n công việc, người thứ i làm việc thứ j với năng suất $s[i,j]$.
Tìm phương án phân công việc cho mỗi người để tổng năng suất lớn nhất.
- Mô tả dữ liệu
 - Input
 - n người được đánh chỉ số $1..n$, n công việc được đánh chỉ số $1..n$
 - Mảng double $s[i:1..n, j:1..n]$ hiệu suất của người i làm việc j
 - Output
 - Double S_{max} hiệu suất công việc cao nhất
 - int $w_{max}[i:1..n]$ công việc phân cho người thứ i (*phương án đạt S_{max}*)
 - Sử dụng biến:
Mảng: boolean $used[j:1..n]$ đánh dấu việc j đã phân công (true) hay chưa.
int $w[i:1..n]$ công việc phân cho người thứ i (*phương án hình thành*)

Example 3

📄 Bài toán: Phân công công việc

```
Try(i) ≡  
    for (v=1..n) //duyet qua các công việc  
        if ((not used[v] && s[i,v]<>-1)) //v chấp nhận được  
            w[i] = v;  
            used[v] = true; //ghi nhận trạng thái đã chọn v  
            if (i=n)  
                Xử_lí_phương_án w[1..n];  
                //Tính tổng hiệu suất, cập nhật Smax, wmax nếu lớn hơn  
            else Try(i+1) ;  
            used[v] = false; //khôi phục trạng thái chưa chọn v  
        endif;  
    endfor;  
  
End.
```

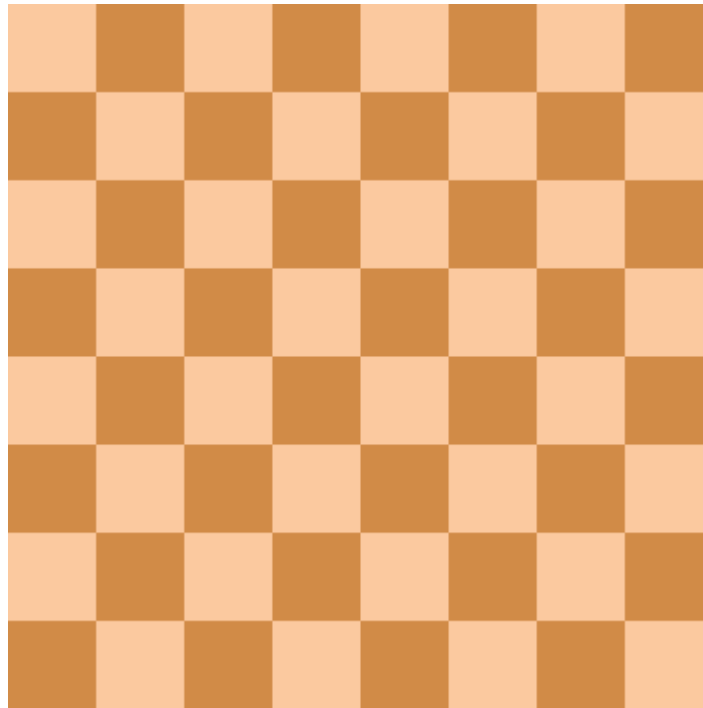
Lời gọi ban đầu: Smax=0; Try(1);

Backtracking applications

- ❑ Hamiltonian Path Problem (Chu trình Hamilton của đồ thị)
 - ❑ Traverling Salesman Problem (Người đưa hàng)
 - ❑ Knapsack Problem (Xếp balo)
 - ❑ Map Coloring Problem (Tô màu bản đồ)
 - ❑ Problem of placing chess (queen, rooks, knights) on chessboard (Bài toán xếp quân cờ: hậu, xe, mã).
 - ❑ ...
-

Backtracking applications

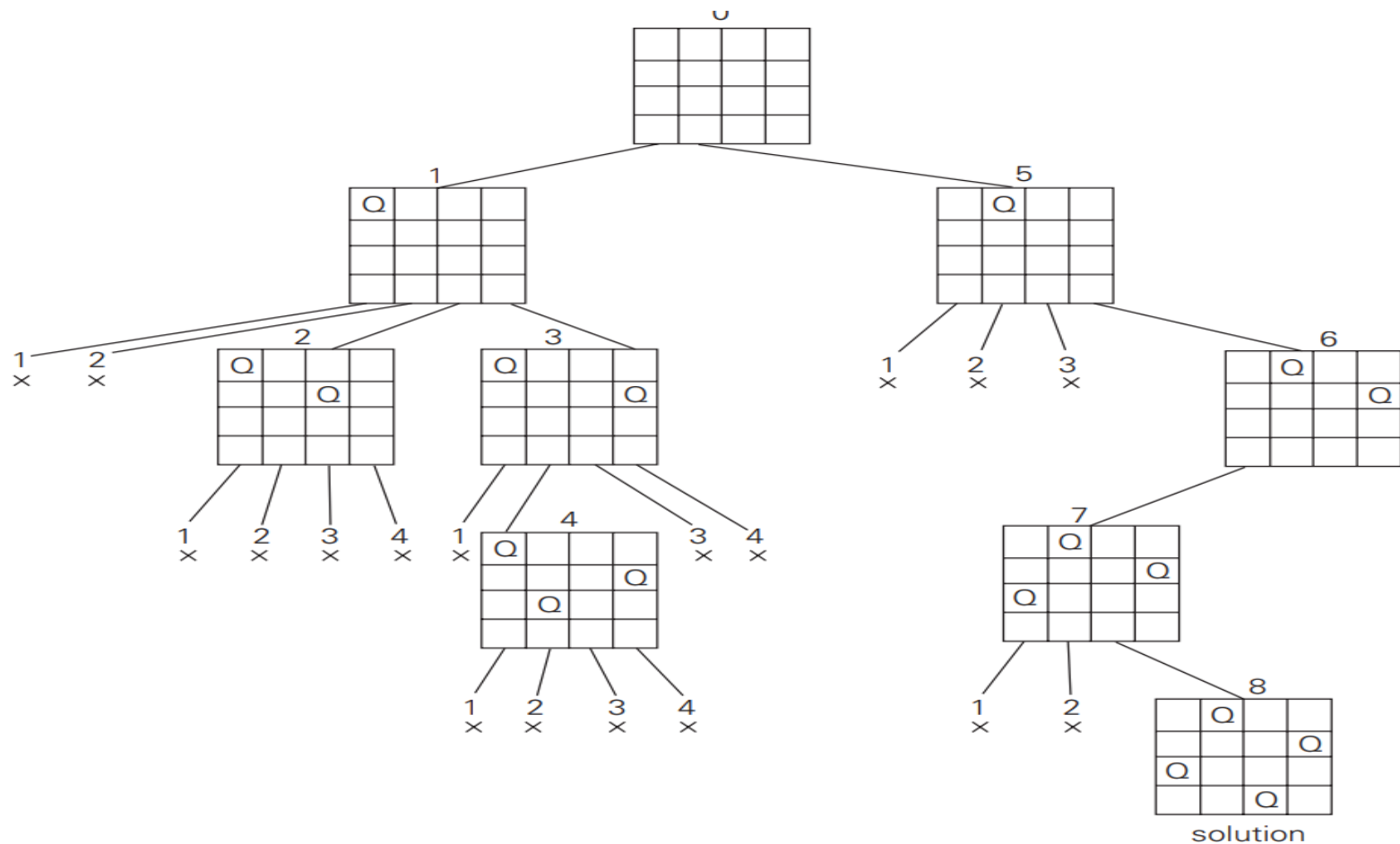
- ❑ Bài toán: Xếp 8 hậu (Eight queens puzzle)



(http://en.wikipedia.org/wiki/Eight_queens_puzzle)

Backtracking applications

Four queens puzzle state-space tree



Branch and Bound

-
- ✓ Introduction
 - ✓ Template and Diagram of Backtracking
 - ✓ Examples
 - ✓ Applications
-

Introduction

□ Bài toán tối ưu

- Bài toán yêu cầu tìm ra một phương án tốt nhất thỏa mãn một số yêu cầu ràng buộc nào đó – nghiệm của bài toán đạt giá trị max/min trong không gian nghiệm.
- Thuộc lĩnh vực Tối ưu toán học hoặc Quy hoạch toán học. Lời giải toán có thể khó => Sự vào cuộc của Tin học
- Hai hướng tiếp cận tìm lời giải tối ưu cho bài toán:
 - Tìm từng lời giải, khi hoàn tất một lời giải thì so sánh chi phí của nó với chi phí tốt nhất hiện có. Nếu tốt hơn thì cập nhật chi phí tốt nhất mới.
 - Với mỗi lời giải, khi xây dựng các thành phần nghiệm luôn kiểm tra điều kiện nếu đi tiếp theo hướng này thì có khả năng nhận được lời giải tốt hơn lời giải hiện có không? Nếu không thì thôi không đi theo hướng này nữa. => Nguyên lý **nhánh cận** (*Branch and Bound*)

Introduction

□ Ý tưởng

- Nhánh cận (Branch and Bound): Thuật toán tìm lời giải cho các bài toán tối ưu dạng liệt kê cấu hình dựa trên nguyên lý đánh giá nhánh cận.
 - **Nguyên lý đánh giá nhánh cận:** Sử dụng các thông tin đã tìm được trong lời giải của bài toán để loại bỏ sớm phương án không dẫn tới lời giải tối ưu
 - Bản chất:
 - Sử dụng phương pháp quay lui nhưng tại mỗi bước đưa thêm thao tác đánh giá giá trị phương án hiện có.
 - Nếu đó là phương án tối ưu hoặc có hy vọng trở thành phương án tối ưu (tức là tốt hơn phương án hiện có) thì cập nhật lại phương án tối ưu hoặc đi tiếp theo hướng đó.
 - Trong trường hợp ngược lại thì bỏ qua hướng đang xét.
-

Introduction

□ Mô hình

Không gian của bài toán (tập khả năng) $D = \{(x_1, x_2, \dots, x_n)\}$ gồm các cấu hình liệt kê có dạng $(x_1, x_2, \dots, x_n)$.

Mỗi cấu hình x sẽ xác định một giá trị hàm chi phí $f(x)$

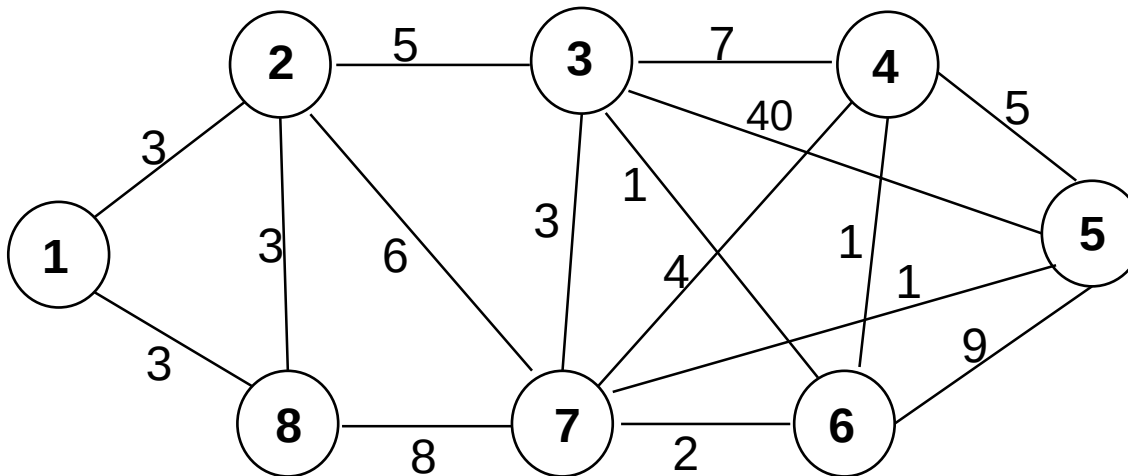
$$f: D \rightarrow Z \quad f(x) = C \quad (C \in Z)$$

Nghiệm của bài toán: $x = (x_1, x_2, \dots, x_n)$ sao cho $f(x) = \text{giá trị tối ưu (max/min)}$

- Cho x_i nhận lần lượt các giá trị có thể. Với mỗi giá trị thử gán cho x_i xét khả năng chọn $x_{i+1}, x_{i+2}, \dots$
- Tại mỗi bước i : Xây dựng thành phần x_i
 - Xác định x_i theo khả năng v .
 - Tính chi phí lời giải nhận được. Nếu “tốt hơn” lời giải hiện thời thì chấp nhận x_i theo khả năng v . Tiếp tục xác định $x_{i+1}, \dots$ đến khi gặp nghiệm
 - Nếu không có một khả năng nào chấp nhận được cho x_i hoặc lời giải xấu hơn thì lùi lại bước trước để xác định lại thành phần x_{i-1} .

Example 1

- Traveling Salesman – Bài toán người đưa hàng
 - Người bán hàng trên hệ thống n thành phố. Giữa các thành phố có thể có hoặc không đường đi có chi phí cho trước. Hãy xác định một hành trình để đi từ một thành phố tới tất cả các thành phố còn lại 1 lần và quay về sao cho tổng chi phí là nhỏ nhất.



Example 1

□ Traveling Salesman:

- Phân tích: Biểu diễn mạng lưới giao thông giữa các thành phố như một đồ thị có trọng số $G=(V,E)$
 - Mỗi thành phố là một nút của đồ thị (đánh số $1,...,n$)
 - Mỗi đường đi giữa các thành phố là một cạnh nối giữa các nút của đồ thị, có thể có hướng hoặc vô hướng, trên đó có ghi trọng số là chi phí đường đi
 - Các cặp cạnh không có đường đi trọng số là ∞
 - Đỉnh xuất phát $\equiv$ kết thúc: $S \in V$
 - Đường đi tìm được là dãy $S=x_1,x_2,...,x_n,x_1=S$ với $x_i \in V, (x_i,x_{i+1}) \in E$, có tổng chi phí nhỏ nhất
- ⇒ Sinh các dãy hoán vị $1..n$ và tính dãy có chi phí nhỏ nhất:
- Quay lui
 - Nhánh cận

Example 1

□ Traveling Salesman:

- Phân tích: Tiếp cận theo phương pháp nhánh cận
 - Chi phí tốt nhất đã tìm được ($BestCost$). Ban đầu $BestCost = +\infty$
 - Tại mỗi bước chọn x_i : Chi phí đường đi từ x_1 đến x_{i-1} là C
 - Với mỗi khả năng v , tính chi phí $C_1 = C +$ chi phí từ x_{i-1} tới v
 - Nếu C_1 xấu hơn (lớn hơn) $BestCost$ hoặc không có khả năng nào chấp nhận được cho x_i thì lùi lại bước trước để xác định lại thành phần x_{i-1} .
 - Nếu C_1 tốt hơn (nhỏ hơn) $BestCost$ thì chấp nhận x_i theo khả năng v . Tiếp tục xác định $x_{i+1}, \dots$
 - Đến khi gặp nghiệm ($i=n+1$ & $x_i=S$):
 - Cập nhật đường đi tốt nhất hiện tại đã tìm được
 - Cập nhật giá trị $BestCost$ mới: $BestCost = C_1$
 - Kết thúc tìm kiếm nếu $BestCost = +\infty \Rightarrow$ không có đường đi

Example 1

□ Traveling Salesman:

■ Mô tả dữ liệu

- Đánh chỉ số các đỉnh của đồ thị từ $1..n$
- Biểu diễn đồ thị G bằng ma trận kề $M = (c_{ij})$ cỡ $n \times n$
 $c_{ij} = \text{cost}$ nếu có cạnh nối đỉnh i với đỉnh j với chi phí cost
 $= \infty$ nếu không có đường đi
- Mảng: $\text{Daqua}[1..n]$ đánh dấu đỉnh i đã được đi qua trên đường đi hay chưa?
 $\text{Daqua}[i] = \text{true}$ nếu đỉnh i đã có trên đường đi
 $= \text{false}$ nếu ngược lại, đỉnh i chưa có trên đường đi
Khởi tạo: $\text{Daqua}[1..n] = \text{false}$

Example 1

□ Traveling Salesman:

```
Try(i,C)≡ //Sinh thành phần thứ i của cấu hình với chi phí hiện thời C
  for (v = 1..n)
    if (c[xi-1,v]<∞) & (not Daqua[v])) )
      C1 = C + c[xi-1,v];
      if (C1 < BestCost) //Tốt hơn chi phí tốt nhất hiện có
        xi = v ;
        Daqua[v] = true;
        if (i = n+1)&(xi = S)
          <Ghi nhận nghiệm x1,x2...xn+1>;
          BestCost = C1; //Cập nhật chi phí tốt nhất
        else if (i<=n)
          Try(i+1,C1) ; //sinh thành phần tiếp theo với chi phí hiện thời C1
        endif;
        Daqua[v] = false;
      endif;
    endif;
  endif;

End.
```

Branch and Bound applications

- ❑ Knapsack problem (Xếp ba lô)
 - ❑ Integer programming (Quy hoạch nguyên)
 - ❑ Nonlinear programming (Quy hoạch phi tuyến)
 - ❑ Maximum satisfiability problem (MAX-SAT - Bài toán thỏa được cực đại)
 - ❑ Nearest neighbor search (NNS - Láng giềng gần nhất)
 - ❑ Cutting stock problem
-

Exercises

- ❑ Program the designed algorithms
 - ❑ Other examples or application
 - ❑ More detail in [Hw5_TrialAndError.doc](#)
-